UNIT AUTHENTICATOR

Implementação de tabela hash com tratamento de colisões por encadeamento

Disciplina	Estrutura de Dados
Linguagem	Java 21
Participante(s)	Lucas Rocha Santos e Lucas Nascimento

1. Objetivo

O sistema armazena pares formados por nome de usuário (chave) e senha (valor). A estrutura de armazenamento foi implementada manualmente, sem HashMap e sem banco de dados. O programa oferece cadastro, remoção, busca e autenticação em um menu de terminal.

2. Organização das classes

Classe	Responsabilidade
Usuario	Representa o par nome e senha. Seus dados são imutáveis após a criação.
Node	Nó da lista ligada: contém um usuário e a referência para o próximo nó.
TabelaHash	Calcula índices e implementa inserção, busca, remoção e autenticação.
Main	Exibe o menu, lê entradas e apresenta o resultado de cada operação.

Fluxo entre os componentes

```
Main -> TabelaHash -> posição do vetor -> Node -> próximo Node. A interface não manipula os nós diretamente; toda a lógica da estrutura fica centralizada em TabelaHash.
```

3. Função hash

O nome é percorrido como uma sequência de caracteres. O acumulador começa em zero e, para cada caractere, é atualizado pela fórmula: hash = hash * 31 + caractere. Ao final, Math.floorMod(hash, tamanho) garante um índice entre zero e tamanho - 1.

```
Exemplo: para uma tabela de tamanho 10, o nome 'ana' produz hash 96724 e índice 4.
```

4. Tratamento de colisões

Cada posição do vetor guarda a cabeça de uma lista ligada. Se dois nomes gerarem o mesmo índice, os dois nós permanecem nessa lista. A inserção coloca o novo nó no início, mantendo uma operação constante.

```
tabela[2] -> [usuário f] -> [usuário a] -> null
```

Na tabela de tamanho 5 usada nos testes, 'a' e 'f' geram o mesmo índice. A remoção de um deles preserva o outro, comprovando o funcionamento do encadeamento.

5. Operações

Operação	Funcionamento	Custo médio
Inserção	Calcula o índice, rejeita duplicidade e liga um novo nó.	O(1)
Busca	Percorre apenas a lista ligada do índice calculado.	O(1)
Login	Busca o nome e compara a senha armazenada.	O(1)
Remoção	Religa o nó anterior ao seguinte, ou atualiza a cabeça.	O(1)

Os custos indicados são médios, assumindo uma boa distribuição das chaves. No pior caso, quando muitas chaves colidem, as operações passam a O(n).

6. Regras adotadas

- nomes e senhas vazios são recusados;
- nomes são únicos e diferenciam maiúsculas de minúsculas;
- a tabela possui 11 posições na execução principal;
- a senha é armazenada como texto somente por exigência didática do enunciado.

7. Testes

Os testes automatizados estão em TabelaHashTest e utilizam JUnit 5. Também foi realizado um teste manual do menu, cobrindo cadastro, busca, login, remoção e encerramento.

Cenário	Resultado esperado
Inserir e buscar usuário	Usuário encontrado
Inserir nome duplicado	Segunda inserção recusada
Login com senha correta	Autenticação aceita
Login com senha incorreta	Autenticação recusada
Remover usuário existente	Usuário deixa de ser encontrado
Remover usuário inexistente	Operação retorna falha
Colisão entre 'a' e 'f'	Remover um preserva o outro
Nome ou senha vazios	IllegalArgumentException

Comandos

```
mvn test
mvn package
java -cp target/classes ink.lucasnsnt.unitauthenticator.Main
```

8. Conclusão

A implementação atende às funcionalidades solicitadas e demonstra os conceitos centrais de tabela hash, função de espalhamento, colisão, lista ligada e remoção de nós. A separação em classes mantém a interface independente da estrutura de armazenamento e facilita a realização de testes.